

Week 1

This lab assignment covers Week 1 (Lectures 1 and 2):

1. Linear Regression 1
2. Train Test Splitting

Submission And Assignment Instructions

Submit your `Lab1_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ? Questions you must answer or tasks are marked with a " ? "
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ? YOUR CODE HERE`
 - ⭐ Bonus questions are marked with a "⭐ "
 - 🍓 Bonus answers are marked with a "🍓 "

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Math!

Answers may be submitted as Latex typeset equations OR inserted as pictures of your handwritten notes. For the later, please ensure you take a clear scan/picture of your readable work. If we can not read your writing, we might have to deduct points for incomplete answers.

1.1: Derivation of the Ordinary Least Squares Estimator with Calculus

In the lecture you were introduced to the OLS equations, let's find out where these equations come from!

? 1.1.1 The mean squared error (MSE) function for a simple linear model can be expressed as $E(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$. Take the partial derivative of E wrt β_0 .

```
In [35]: from IPython.display import Image, display
display(Image("1.1.1.png"))
```

1.1.1

$$E(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

$$\frac{\partial E}{\partial \beta_0} = \sum_{i=1}^n 2(y_i - \beta_0 - \beta_1 x_i)(-1) = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)$$

? 1.1.2 Take the partial derivative of E wrt β_1 .

In [36]: `from IPython.display import Image, display`
`display(Image("1.1.2.png"))`

$$\frac{\partial E}{\partial \beta_1} = \sum_{i=1}^n 2(y_i - \beta_0 - \beta_1 x_i)(-x_i) = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)(x_i) = -2 \sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2)$$

? 1.1.3 Set both partial derivatives equal to 0 and solve for the optimal parameter values of β_0 and β_1 . You must show your work.

In [37]: `from IPython.display import Image, display`
`display(Image("1.1.3.png"))`

<u>1.1.3</u> $\frac{\partial E}{\partial \beta_0} = 0$ $\underbrace{-2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)}_{-2} = 0$ $\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$ $\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \beta_1 \sum_{i=1}^n x_i = 0$ $n\bar{y} - n\beta_0 - n\beta_1 \cdot n\bar{x} = 0$ $n(\bar{y} - \beta_1 \bar{x}) = n\beta_0$ $\beta_0 = \bar{y} - \beta_1 \bar{x}$	$\frac{\partial E}{\partial \beta_1} = 0$ $\underbrace{-2 \sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2)}_{-2} = 0$ $\sum_{i=1}^n x_i y_i - \beta_0 \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 = 0$ sub in β_0 $\sum_{i=1}^n x_i y_i - \sum_{i=1}^n (\bar{y} - \beta_1 \bar{x}) x_i - \sum_{i=1}^n \beta_1 x_i^2 = 0$ $\sum_{i=1}^n x_i y_i - \bar{y} \sum_{i=1}^n x_i + \beta_1 \bar{x} \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 = 0$ $\underbrace{\sum_{i=1}^n x_i y_i - n\bar{y}\bar{x}}_{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})} + \beta_1 (n\bar{x}^2 - \sum_{i=1}^n x_i^2) = 0$ $\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{(n\bar{x}^2 - \sum_{i=1}^n x_i^2)}$ $\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$
---	---

★ 1.2: Bonus - Linear Algebra Notation for Multiple Linear Regression

We can rewrite the OLS equations above like this:

$$\hat{Y} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \quad (1)$$

$$= \begin{pmatrix} 1 & | \\ \vdots & \vec{x} \\ 1 & | \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \quad (2)$$

$$= X\vec{\beta} \quad (3)$$

★ 1.2.1 Why do we need a column of 1s in the X matrix above? (one sentence) \

🍓 Answer: To account for the y-intercept.

★ 1.2.2 Imagine we now have 2 independent variables, which we'll call $\vec{x}_1$ and $\vec{x}_2$, that we wish to use to predict $\vec{y}$. We will also fit a new parameter, β_2 to represent the "slope" from

the X_2 predictor. Write the new equations for $\hat{Y}$ in matrix form, with $\vec{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$.

In [38]: `from IPython.display import Image, display`
`display(Image("1.2.2.png"))`

1.2.2

$$\hat{Y} = \begin{pmatrix} 1 & x_{1,1} & x_{2,1} \\ 1 & x_{1,2} & x_{2,2} \\ \vdots & \vdots & \vdots \\ 1 & x_{1,n} & x_{2,n} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$$

★ 1.2.3 Write new equations for $\vec{e}$, the vector of residuals in matrix form, with $\vec{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$.

In [39]: `from IPython.display import Image, display`
`display(Image("1.2.3.png"))`

1.2.3

$$\vec{e} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} 1 & x_{1,1} & x_{2,1} \\ 1 & x_{1,2} & x_{2,2} \\ \vdots & \vdots & \vdots \\ 1 & x_{1,n} & x_{2,n} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix} = \vec{y} - X\vec{\beta}$$

Section 2: Brain and Body Weights

2.1: Loading the data

Use the provided code block to upload the file to collab.

```
In [40]: # load local file "data01.txt"
from google.colab import files

uploaded = files.upload()

for fn in uploaded.keys():
    print('User uploaded file "{name}" with length {length} bytes'.format(
        name=fn, length=len(uploaded[fn])))
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[40], line 2
      1 # load local file "data01.txt"
----> 2 from google.colab import files
      4 uploaded = files.upload()
      6 for fn in uploaded.keys():

ModuleNotFoundError: No module named 'google'
```

? 2.1.1 Write code to read in your uploaded file as a pandas dataframe and visualize the first 5 rows.

```
In [41]: # import numpy and pandas to prep your workspace
import numpy as np
import pandas as pd

## ? YOUR CODE HERE
# Read data in as a pandas df
data = pd.read_csv('Lab01_Data_01_brains.txt', sep='\t', index_col=0)

# visualize first five rows using data.head()
data.head(5)
```

```
Out[41]:
```

	Body.Weight	Brain.Weight
Index		
1	3.385	44.5
2	0.480	15.5
3	1.350	8.1
4	465.000	423.0
5	36.330	119.5

? 2.1.2 Identify and describe the variables in this dataframe. Are these categorical? Ordinal? Integer? Logical?

✓ Answer: The variables in this dataframe are the following: Index, Body.Weight and Brain.Weight. Index is an integer, while Body.Weight and Brain.Weight are numeric variables.

2.2: Data Exploration & Visualization

? 2.2.1 Let's predict brain weight from body weight. Visualize brain weight and body weight on a plot, with the independent variable on the x-axis. Be sure to label all parts of your figure (axis labels with units and title).

```
In [42]: # first import matplotlib
from matplotlib import pyplot as plt

plt.figure()
plt.scatter(data['Body.Weight'], data['Brain.Weight'])
plt.xlabel('Body Weight (kg)')
plt.ylabel('Brain Weight (g)')
plt.title('Brain Weight vs Body Weight')
plt.show()
```



? 2.2.2 What are the assumptions for linear regression? Describe each assumption with <1 sentence.

✓ Answer:

1. Linearity: x and y must have a linear relationship.
2. Residuals are normally distributed and independent.
3. Homoscedacity: the variance of the residuals is the same for any value of x.

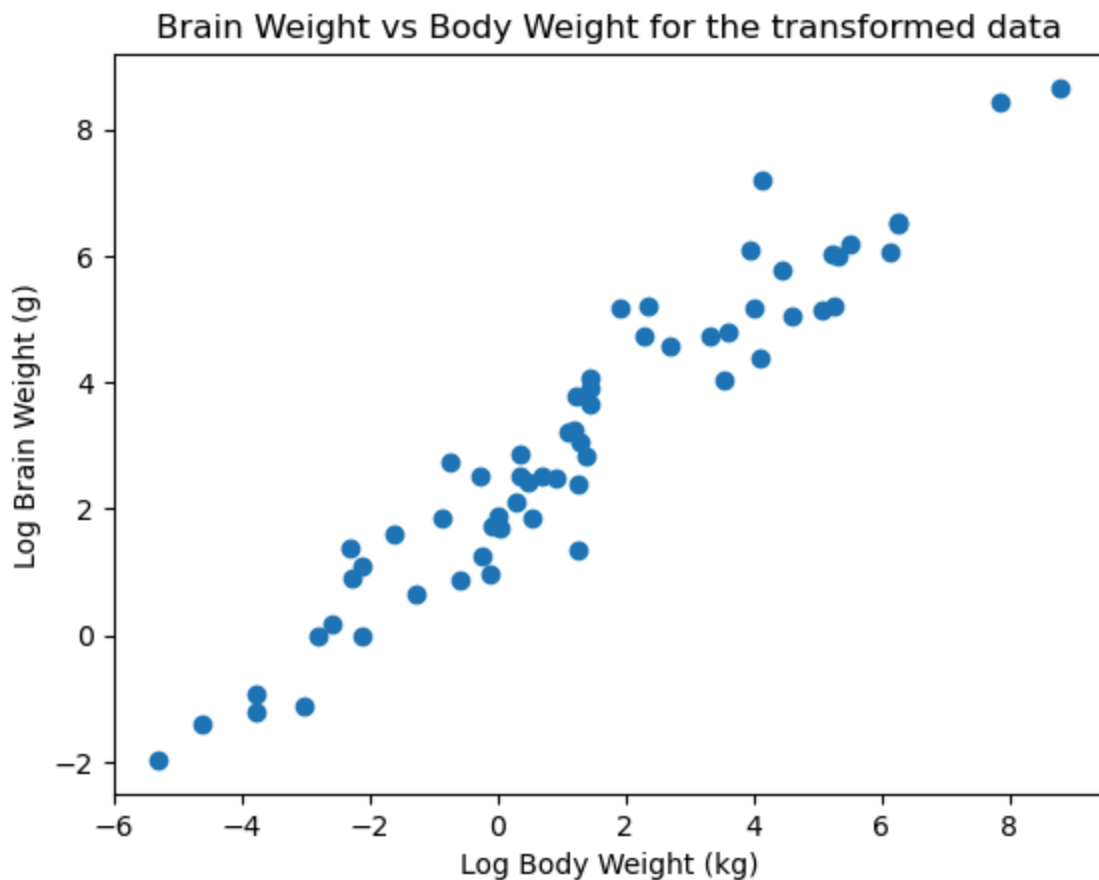
2.3: Transformation

? 2.3.1 Transform the data to try to correct the linearity assumption and replot. Label your plot appropriately (include axis labels, title, units).

```
In [43]: datalog = data.assign(
    BodyWeight_log = np.log(data['Body.Weight']),
    BrainWeight_log = np.log(data['Brain.Weight'])
)

datalog.head(5)

plt.figure()
plt.scatter(datalog['BodyWeight_log'], datalog['BrainWeight_log'])
plt.xlabel('Log Body Weight (kg)')
plt.ylabel('Log Brain Weight (g)')
plt.title('Brain Weight vs Body Weight for the transformed data')
plt.show()
```



✓ Answer: From visualization we can observe that x and y have a linear relationship. Additionally, we can observe that the residuals for any value of x are similar which satisfies the homoscedasticity assumption. However, we cannot seem to tell if the residuals are normally distributed and independent by looking at the plot.

2.4: A Simple Linear Model

? 2.4.1 Calculate the intercept and slope of a line of best fit on the transformed data using the formulas you derived above (also in the slides from class!)

```
In [44]: ## ? YOUR CODE HERE

# calculate slope
x = datalog['BodyWeight_log']
y = datalog['BrainWeight_log']
x_bar = x.mean()
y_bar = y.mean()
m = ((x - x_bar) * (y - y_bar)).sum() / ((x - x_bar) ** 2).sum()

# be nice to your TAs
# try to print outputs from code labelled and easy to read
# this is an example if you name your slope variable m
print("Slope: ", m)
```

Slope: 0.7516859362419007

```
In [45]: ## ? YOUR CODE HERE
# calculate and display intercept
b = y_bar - m * x_bar
# display and be nice to the TAs
print("Intercept: ", b)
```

Intercept: 2.134788676764636

2.5: Assessing the Model

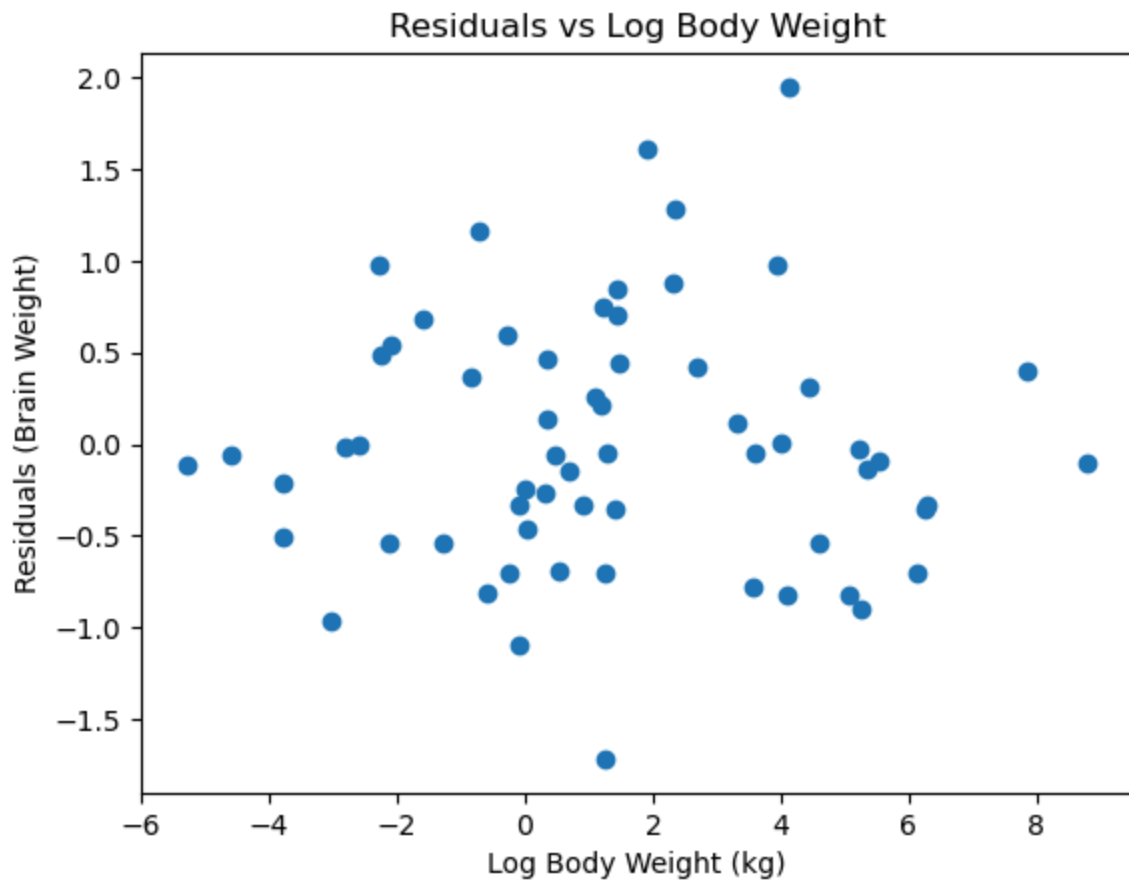
Visualizing Error

To do linear regression, we assume that there is a linear relationship between x and y , and we assume that the errors in y are normally and equally distributed at each x value. One of the easiest ways to visualize whether these assumptions hold in our model is to make a residuals plot, which displays the errors of the model (the residuals, $y - \hat{y}$) against the x -values themselves.

? 2.5.1 Calculate residuals and add them to your pandas dataframe. Make a residuals plot and label appropriately (include axis labels, units, plot title)

```
In [46]: ## ? YOUR CODE HERE
# calculate residuals for each observation
datalog = datalog.assign(
    residuals = datalog['BrainWeight_log'] - (m * datalog['BodyWeight_log'] + b)
)

# Make a residuals plot using matplotlib
plt.figure()
plt.scatter(datalog['BodyWeight_log'], datalog['residuals'])
plt.xlabel('Log Body Weight (kg)')
plt.ylabel('Residuals (Brain Weight)')
plt.title('Residuals vs Log Body Weight')
plt.show()
```

? 2.5.2 Briefly interpret your residuals plot. Do you see any patterns? What do the patterns (or lack thereof) imply about your model?

✓ Answer: The residuals form a cloud with no obvious pattern which satisfies the independence assumption. Moreover, the residuals are clustered around zero, roughly suggesting that the residuals are normally distributed.

2.6 Visualize Model

? 2.6.1 Add a line of best fit to your log-scaled data plot from earlier.

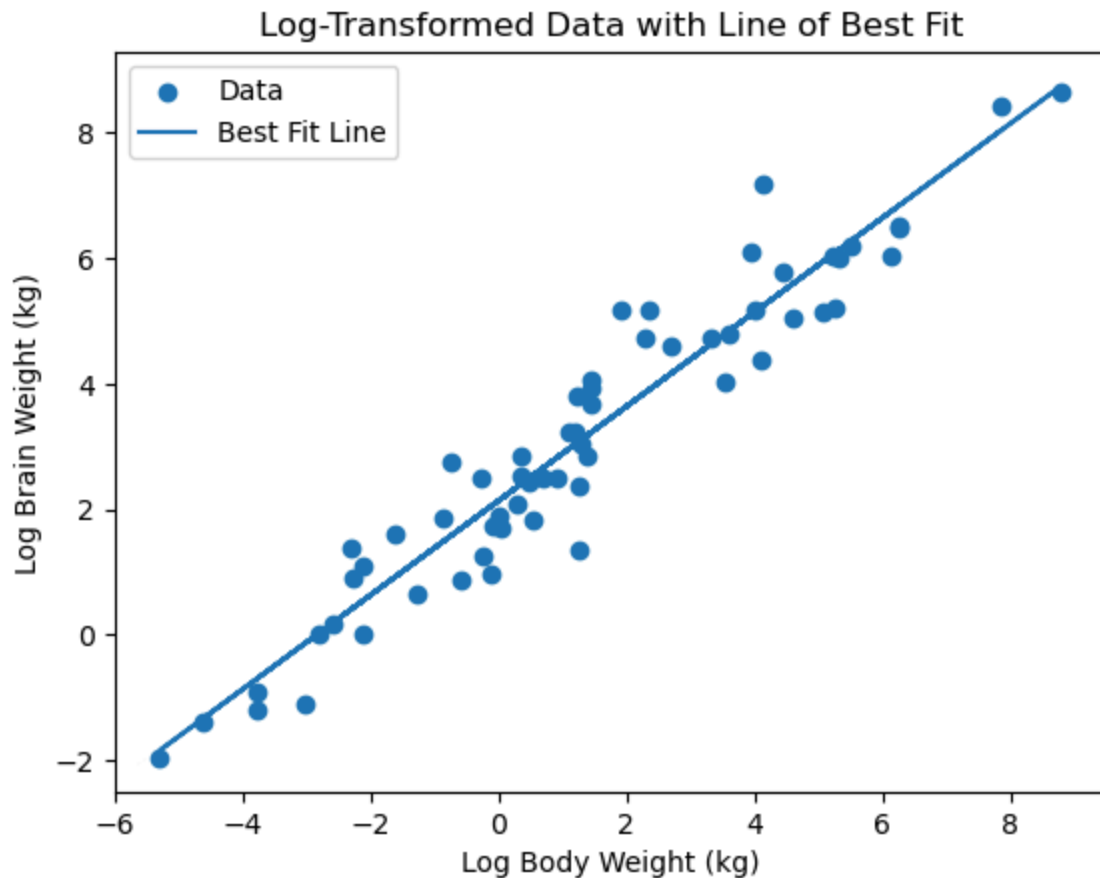
```
In [47]: ## ? YOUR CODE HERE
# visualize the transformed data
fig, ax = plt.subplots()

ax.scatter(
    datalog['BodyWeight_log'],
    datalog['BrainWeight_log'],
    label='Data'
)

ax.plot(
    datalog['BodyWeight_log'],
    m * datalog['BodyWeight_log'] + b,
    label='Best Fit Line'
)
```

```
ax.set_xlabel('Log Body Weight (kg)')
ax.set_ylabel('Log Brain Weight (kg)')
ax.set_title('Log-Transformed Data with Line of Best Fit')

# remember to add a legend
ax.legend()
plt.show()
```



★ 2.7 Bonus - Quantitative Error Assessments

Please do this *after* finishing the required assignment.

★ 2.7.1 Calculate R^2 using the equation from class.

```
In [49]: ## ★ YOUR CODE HERE
# Calculate and display R2

# calculate slope & intercept of raw data
x_raw = datalog['Body.Weight']
y_raw = datalog['Brain.Weight']
x_bar_raw = x_raw.mean()
y_bar_raw = y_raw.mean()
m_raw = ((x_raw - x_bar_raw) * (y_raw - y_bar_raw)).sum() / ((x_raw - x_bar_raw)**2).sum()
b_raw = y_bar_raw - m_raw * x_bar_raw
# Calculate and display R2 for raw data
SSres = ((datalog['Brain.Weight'] - (m_raw * datalog['Body.Weight'] + b_raw))**2).sum()
SStot = ((datalog['Brain.Weight'] - y_bar_raw)**2).sum()
R2_rawdata = 1 - (SSres/SStot)
print("Coefficient of determination: ", R2_rawdata,
```

```

        "\nSSres: ", SSres,
        "\nSStot: ", SStot)

print()

#Calculate and display R2 for the log model
SSres_ln = ((datalog['BrainWeight_log'] - (m * datalog['BodyWeight_log'] + b)
SStot_ln = ((datalog['Brain.Weight'] - (y_bar)**2)).sum()
R2_ln = 1 - (SSres_ln / SStot_ln)
print("Coefficient of determination for the natural log model: ", R2_ln,
      "\nSSres: ", SSres_ln,
      "\nSStot: ", SStot_ln)

```

```

Coefficient of determination:  0.8726620843043331
SSres:  6722239.055504983
SStot:  52790553.534509696

```

```

Coefficient of determination for the natural log model:  0.9982929351570395
SSres:  28.92271042146065
SStot:  16942.9477390573

```

★ 2.7.2 Explain what you calculated and how it connects to the interpretation of R^2 .
(What's the total sum of squared errors? Why do we need to use it?)

🍓 : I calculated the coefficient of determination R^2 for both the raw data model and the log-transformed model. It is computed as $1 - (SSres / SStot)$. SSres is the residual sum of squares, representing the unexplained variation after fitting the model, and SStot is the total sum of squares, representing the total variability of the data around its mean. We need the total sum of squared errors because it provides a baseline for comparison, allowing us to determine how much of that total variability is reduced by the model. Overall, R^2 quantifies how well the model explains the data, with larger values indicating a better fit.

★ 2.7.3 Does it matter if we calculate R^2 from the log-scaled data or from the raw (untransformed) data of body/brain weights? Re-calculate R^2 from the untransformed data and estimates of brain weight from your model and display it. Does this estimate match what you calculated above? Why or why not?

```

In [50]: ##★ YOUR CODE HERE
# recalculate some of the quantities you found above (SSres, SStot, R^2)
# and compare to what you had before
SSres = ((datalog['Brain.Weight'] - (m_raw * datalog['Body.Weight'] + b_raw))**2).sum()
SStot = ((datalog['Brain.Weight'] - (y_bar_raw)**2)).sum()
R2_rawdata = 1 - (SSres/SStot)
print("Coefficient of determination: ", R2_rawdata,
      "\nSSres: ", SSres,
      "\nSStot: ", SStot)

print()

# doing the same for the natural log model
SSres_ln = ((datalog['BrainWeight_log'] - (m * datalog['BodyWeight_log'] + b)
SStot_ln = ((datalog['Brain.Weight'] - (y_bar)**2)).sum()
R2_ln = 1 - (SSres_ln / SStot_ln)
print("Coefficient of determination for the natural log model: ", R2_ln,

```

```
"\nSSres: ", SSres_ln,
"\nSStot: ", SStot_ln)
```

Coefficient of determination: 0.8726620843043331
 SSres: 6722239.055504983
 SStot: 52790553.534509696

Coefficient of determination for the natural log model: 0.9982929351570395
 SSres: 28.92271042146065
 SStot: 16942.9477390573

🍓 : The coefficient of determination for the natural log model is higher, indicating that it explains a larger proportion of the variability because it provides a better fit to the data than the raw-data model.

⭐ 2.7.4 Calculate MSE and RMSE explicitly and display their values.

```
In [51]: ## ⭐ YOUR CODE HERE
# calculate MSE and RMSE
n = len(data)
MSE = SSres / n
RMSE = np.sqrt(MSE)
# display MSE and RMSE
print("MSE: ", MSE, "\nRMSE: ", RMSE)

print()
# doing the same for ln
# calculate MSE and RMSE
# calculate MSE and RMSE
n_ln = len(data_log)
MSE_ln = SSres_ln / n_ln
RMSE_ln = np.sqrt(MSE_ln)
# display MSE and RMSE
print("MSE for natural log model: ", MSE_ln, "\nRMSE for natural log model: ",
MSE: 108423.21057266102
RMSE: 329.27679932339754
```

MSE for natural log model: 0.4664953293783976
 RMSE for natural log model: 0.6830046334970192

2.8: Model Interpretation

? 2.8.1 This model was built on data from 62 mammalian species. Does it do a good job predicting species brain weight from body weight? Suggest one way to make this model better.

✅ Answer: Overall, the model does a reasonably good job predicting species brain weight from body weight, as evidenced by the strong linear trend and the relatively small residuals around the fitted line. However, some variability remains unexplained, indicating that body weight alone does not fully capture differences in brain weight across species. One way to improve the model would be to include additional biologically relevant variables, (such as metabolic rate) to account for systematic differences between species

? 2.8.2 Is this model a good way to predict human brain weights from body weights?

✓ Answer: No because human brain weight is disproportionately large for our body size compared to most other mammals, so a model fit to mammalian data underestimates human brain weight.

Feedback

? What's one thing you learned in this assignment?

✓ Answer: How to code using pandas

? What do you wish had been different about this assignment?

✓ Answer: Maybe a bit more guidance on how to code, for example when it says use `data.assign` maybe give more details + what it does. Else than that it was great!